



Rapport — Optimisation Convexe

Sujet A : Comparaison des optimiseurs sur CIFAR-10
SGD, SGD+Momentum, Adam, RMSProp, Moun

Thi Tho PHAM Alexan LEBLOIS
Gabriel ORCUN Ilyann MOUISSET-FERRARA

EFREI Paris — Cours d'Optimisation Convexe

Juin 2026

Résumé

Ce rapport présente une étude comparative des principaux algorithmes d'optimisation utilisés pour l'entraînement de réseaux de neurones convolutifs sur le jeu de données CIFAR-10. Nous analysons le comportement de cinq optimiseurs — SGD, SGD avec momentum, Adam, RMSProp et Moun — selon plusieurs critères : vitesse de convergence, précision finale, généralisation et robustesse aux hyperparamètres. Une attention particulière est portée à la nature non convexe du paysage de perte des réseaux profonds, et à la façon dont chaque optimiseur y répond.

Table des matières

1	Introduction	3
2	Fondementals des CNNs et des optimiseurs	3
2.1	Réseaux de neurones convolutifs et apprentissage profond	3
2.2	Descente de gradient stochastique (SGD)	3
2.3	SGD avec momentum	3
2.4	Muon	4
2.5	RMSPprop	4
2.6	Adam	4
2.7	Non-convexité et implications pratiques	4
3	État de l’art sur CIFAR-10	5
4	Protocole expérimental	5
4.1	Architecture	5
4.2	Données et prétraitement	5
4.3	Optimiseurs et hyperparamètres	6
4.4	Métriques et reproductibilité	6
5	Résultats et analyse	7
5.1	Courbes de convergence	7
5.2	Précision finale et vitesse de convergence	9
5.3	Sensibilité au taux d’apprentissage	10
5.4	Normes des gradients	10
6	Discussion	10
6.1	Lien avec la non-convexité	10
6.2	Convergence et généralisation	11
6.3	Limites de l’étude	11
7	Conclusion	11

1 Introduction

Dans ce projet, nous choisissons d'utiliser un réseau de neurones convolutif (CNN) pour résoudre cette tâche de classification. Les CNNs se sont imposés comme l'architecture de référence pour la vision par ordinateur grâce à leur capacité à extraire automatiquement des caractéristiques visuelles hiérarchiques — des contours et textures aux formes complexes — directement depuis les pixels bruts. Notre objectif n'est pas de maximiser la précision du modèle, mais d'utiliser ce cadre pour observer et comparer le comportement de cinq optimiseurs : SGD, SGD avec momentum, Muon, RMSProp et Adam.

2 Fondementals des CNNs et des optimiseurs

2.1 Réseaux de neurones convolutifs et apprentissage profond

L'apprentissage profond (*deep learning*) s'agit l'entraînement de réseaux de neurones à plusieurs couches, capables d'apprendre automatiquement des représentations hiérarchiques à partir des données brutes. Les CNNs est un l'architecture de référence pour la vision par ordinateur.

Un CNN est composé de trois types de couches : les couches de convolution qui extraient des motifs visuels locaux, les couches de pooling qui réduisent la dimension spatiale, et les couches fully-connected qui produisent la décision finale de classification.

L'entraînement des réseaux revient à minimiser une fonction de coût $\mathcal{L}(\theta)$ non convexe. Nous comparons cinq algorithmes : SGD, SGD avec momentum, Muon, RMSProp et Adam

2.2 Descente de gradient stochastique (SGD)

La descente de gradient stochastique, pour chaque itération, les paramètres θ sont mis à jour dans la direction opposée au gradient de la fonction de coût :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1)$$

où $\eta > 0$ est le taux d'apprentissage et $\nabla_{\theta} \mathcal{L}$ est le gradient estimé sur un mini-batch. La convergence peut être lente lorsque la surface de perte présente des courbures très différentes selon les directions.

2.3 SGD avec momentum

Le SGD avec momentum utilise un vecteur de vitesse pour la mise à jour de base, ce qui permet de conserver une direction de descente cohérente d'une itération à l'autre :

$$v_{t+1} = \mu v_t + \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2)$$

$$\theta_{t+1} = \theta_t - v_{t+1} \quad (3)$$

Le coefficient $\mu \in [0, 1)$ (typiquement 0,9) contrôle la contribution des gradients passés. Cette mémoire accélère la convergence dans les directions de gradient stable.

2.4 Muon

Muon [6] est un optimiseur récent conçu pour les matrices de poids des couches cachées. Il applique une étape de Newton-Schulz pour orthogonaliser le gradient avant la mise à jour, ce qui revient à mettre à jour les poids dans la direction du gradient le plus proche selon la distance spectrale :

$$\theta_{t+1} = \theta_t - \eta \text{NewtonSchulz}(G_t) \quad (4)$$

Cette orthogonalisation garantit que chaque direction de mise à jour est normalisée spectralement, ce qui stabilise l'entraînement et permet d'utiliser des taux d'apprentissage plus élevés sans divergence.

2.5 RMSProp

RMSProp [7] reprend l'idée d'Muon mais remplace la somme cumulative par une moyenne mobile exponentielle, ce qui évite que le taux d'apprentissage ne s'annule avec le temps :

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2 \quad (5)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t \quad (6)$$

avec $\rho \approx 0,9$. Seuls les gradients récents influencent la mise à l'échelle, ce qui rend RMSProp plus stable et efficace pour l'entraînement de réseaux profonds.

2.6 Adam

Adam (Adaptive Moment Estimation) [5] combine les idées du momentum (premier moment) et de l'adaptivité de RMSProp (second moment) :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (8)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} m_t \quad (9)$$

avec $\beta_1 = 0,9$, $\beta_2 = 0,999$ et $\epsilon = 10^{-8}$ par défaut. Adam ajoute une correction de biais sur m_t et v_t pour les premières itérations. Sa robustesse au choix du taux d'apprentissage en fait un optimiseur pratique et largement adopté.

2.7 Non-convexité et implications pratiques

Les fonctions de coût des réseaux profonds sont non convexes : elles présentent des minima locaux, des points-selles et des plateaux qui compliquent l'optimisation. En pratique, les optimiseurs basés sur le gradient ne garantissent pas de trouver le minimum global, mais convergent vers des solutions acceptables grâce au bruit stochastique et à la haute dimensionnalité de l'espace des paramètres. Le choix de l'optimiseur influence non seulement la vitesse de convergence, mais aussi la qualité des minima trouvés et donc la capacité du modèle à généraliser.

3 État de l’art sur CIFAR-10

CIFAR-10 est un jeu de données classique pour évaluer les réseaux de neurones convolutifs. Les meilleurs résultats publiés sont généralement obtenus avec des architectures plus profondes que celle utilisée dans ce rapport, comme VGG, ResNet ou DenseNet, combinées à de longues phases d’entraînement, de l’augmentation de données et un réglage précis des hyperparamètres [2, 3, 4]. Ces travaux servent de repères, mais ils ne sont pas directement comparables à notre protocole, qui utilise volontairement un CNN plus simple afin d’isoler l’effet du choix de l’optimiseur.

Dans la littérature, SGD avec momentum reste une référence solide pour les tâches de vision. Les méthodes adaptatives comme Adam ou RMSProp sont souvent appréciées parce qu’elles réduisent rapidement la perte et demandent moins de réglage manuel du taux d’apprentissage. Cependant, Wilson et al. [8] montrent que cette convergence plus rapide ne garantit pas toujours une meilleure généralisation : dans certains cas, SGD avec momentum atteint de meilleures performances finales que les optimiseurs adaptatifs. Cette différence est généralement liée aux trajectoires suivies dans le paysage non convexe et aux minima atteints par chaque méthode.

L’objectif de notre expérience n’est donc pas de battre l’état de l’art sur CIFAR-10, mais de comparer, dans un cadre contrôlé, la vitesse de convergence, la précision finale et l’écart de généralisation de plusieurs optimiseurs sur une même architecture.

Conclusion courte. Cette section montre que les performances de référence sur CIFAR-10 dépendent fortement de l’architecture et du protocole d’entraînement. Dans notre cas, la comparaison porte donc surtout sur le comportement relatif des optimiseurs : les méthodes adaptatives peuvent accélérer l’apprentissage, tandis que SGD avec momentum reste une référence importante pour analyser la généralisation.

4 Protocole expérimental

4.1 Architecture

Nous utilisons un CNN simple composé de trois blocs convolutifs suivis de deux couches fully-connected. Chaque bloc convolutif contient une convolution, une activation ReLU et une couche de max-pooling. Les canaux passent successivement de 3 à 32, puis 64 et 128. Après les trois blocs, les cartes de caractéristiques sont aplaties, puis envoyées dans une couche linéaire de 256 neurones avant la couche finale de classification à 10 sorties, correspondant aux 10 classes de CIFAR-10.

4.2 Données et prétraitement

CIFAR-10 contient 50 000 images d’entraînement et 10 000 images de test, réparties équitablement sur 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). Les augmentations appliquées sont : retournement horizontal aléatoire, recadrage aléatoire (padding=4), normalisation par la moyenne et l’écart-type du jeu d’entraînement.

4.3 Optimiseurs et hyperparamètres

Nous n’effectuons pas de recherche exhaustive du meilleur taux d’apprentissage. Chaque optimiseur est entraîné avec un taux d’apprentissage initial choisi à partir de valeurs usuelles, puis ce taux est diminué progressivement avec un scheduler.

CosineAnnealingLR. Cette approche permet de comparer les optimiseurs dans un cadre identique tout en évitant de multiplier les entraînements pour une grille de valeurs.

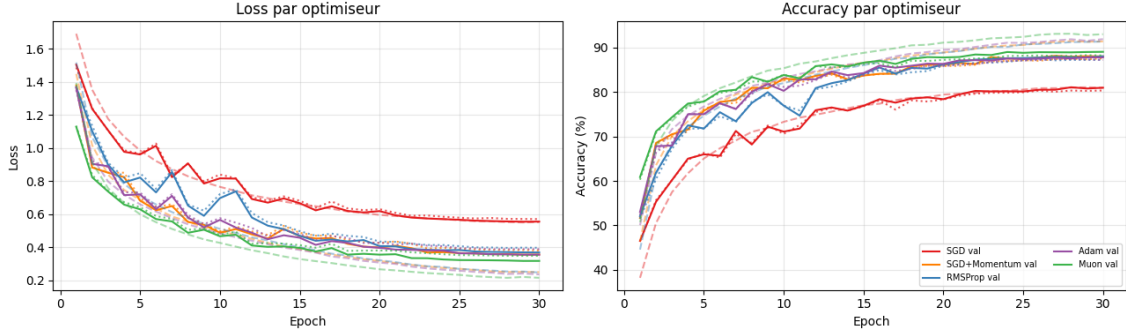


FIGURE 1 – Évolution du taux d’apprentissage par époque

TABLE 1 – Hyperparamètres des optimiseurs utilisés

Optimiseur	Paramètres principaux	LR initial
SGD	Aucun momentum	10^{-2}
SGD avec momentum	$\mu = 0.9$	10^{-2}
Muon	Orthogonalisation Newton-Schulz	10^{-2}
RMSProp	$\rho = 0.9, \epsilon = 10^{-8}$	1.39×10^{-3}
Adam	$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$	1.39×10^{-3}

4.4 Métriques et reproductibilité

Pour comparer les optimiseurs de manière cohérente, nous utilisons les mêmes métriques pour chaque expérience. La performance est évaluée principalement par l’accuracy sur les ensembles d’entraînement, de validation et de test. Nous suivons également la précision macro, qui mesure la qualité des prédictions en tenant compte des dix classes de CIFAR-10 de manière équilibrée.

En plus des métriques de classification, nous analysons la perte d’entraînement et de test afin d’observer la vitesse de convergence de chaque optimiseur. L’écart de généralisation est calculé comme la différence entre l’accuracy d’entraînement et l’accuracy de test :

$$\text{Gap}_{\text{gén.}} = \text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{test}}. \quad (10)$$

Un écart élevé indique que le modèle s’adapte davantage aux données d’entraînement, tandis qu’un écart faible suggère une meilleure capacité de généralisation.

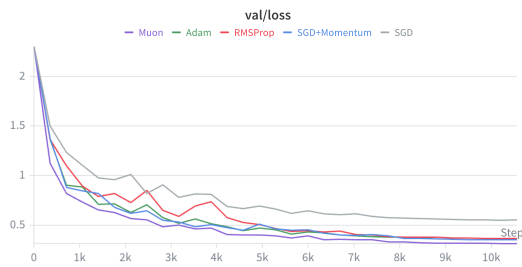
Afin de rendre les résultats reproductibles, nous fixons la graine aléatoire à 42 pour PyTorch et NumPy. Tous les optimiseurs sont entraînés avec la même architecture, le même découpage des données, le même nombre d’époques (30), la même taille de batch (128) et le même scheduler de taux d’apprentissage. Les différences observées proviennent donc principalement du choix de l’optimiseur.

5 Résultats et analyse

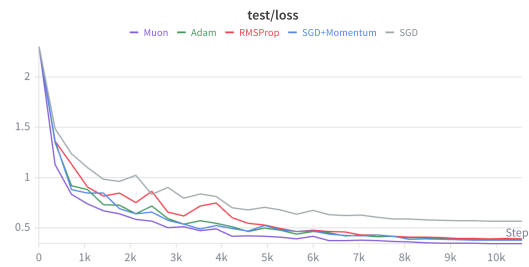
5.1 Courbes de convergence



(a) Courbes de loss sur l'entraînement



(b) Perte de validation



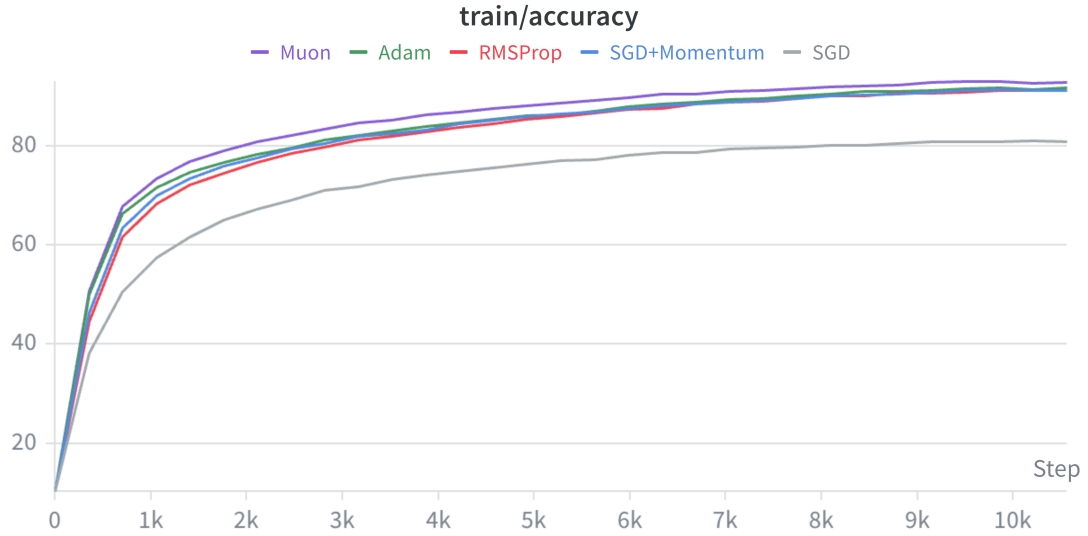
(c) Perte de test

FIGURE 2 – Courbes de perte pour les cinq optimiseurs (30 époques)

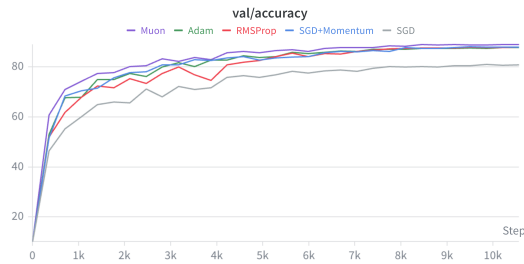
La figure 2 montre que tous les optimiseurs réduisent rapidement la perte au début de l'entraînement, puis convergent plus lentement après les premières époques. La différence principale concerne la vitesse de descente et le niveau final atteint. Muon obtient la diminution de perte la plus rapide et termine avec la perte la plus faible sur les ensembles d'entraînement, de validation et de test.

Adam, RMSProp et SGD avec momentum ont des comportements proches. Ces trois méthodes convergent nettement plus vite que SGD seul et atteignent des pertes finales similaires. RMSProp présente cependant davantage d'oscillations au début sur la validation et le test, ce qui indique une trajectoire moins régulière.

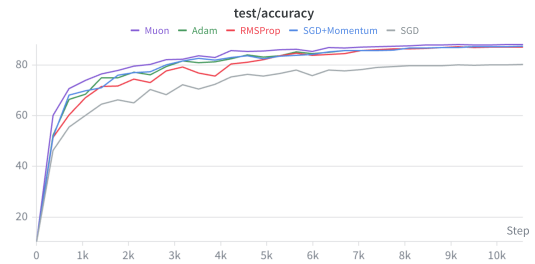
SGD seul est l'optimiseur le plus lent. Sa perte diminue de manière stable, mais elle reste nettement plus élevée que celle des autres méthodes pendant toute l'expérience. Cela montre que, dans ce cadre, l'absence de momentum ou d'adaptation du taux d'apprentissage ralentit fortement la convergence.



(a) Accuracy d'entraînement



(b) Accuracy de validation



(c) Accuracy de test

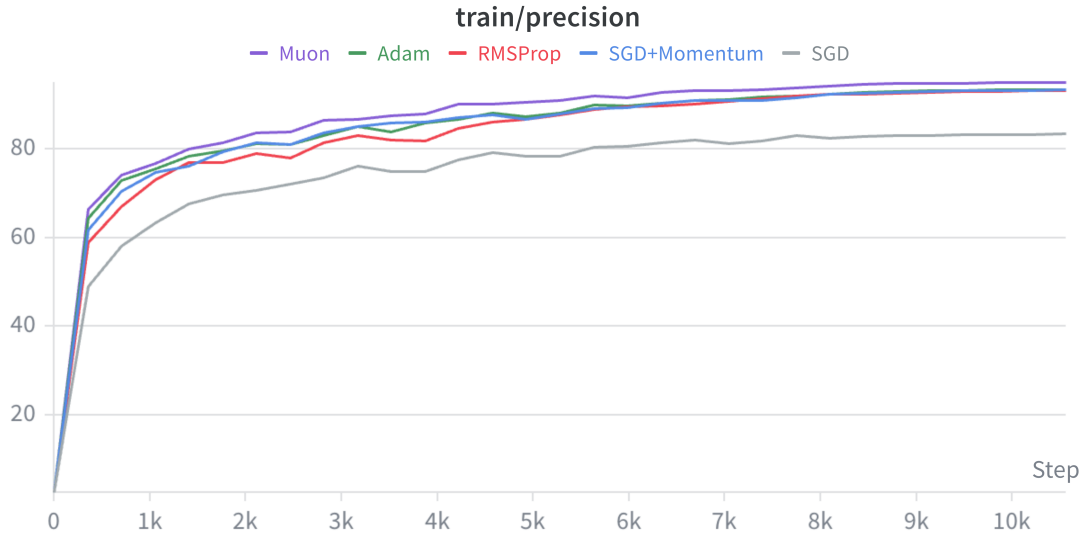
FIGURE 3 – Courbes d'accuracy pour les cinq optimiseurs (30 époques)

Muon atteint rapidement une accuracy élevée et conserve un léger avantage sur la validation et le test jusqu'à la fin de l'entraînement. Il obtient aussi la meilleure accuracy d'entraînement, ce qui indique une optimisation très efficace du modèle.

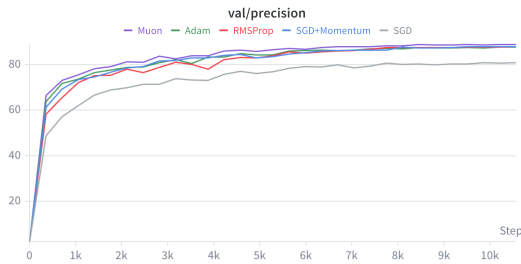
Adam, RMSProp et SGD+Momentum forment un groupe intermédiaire très proche. Ces particulièrement stables après les premières époques, tandis que RMSProp montre quelques variations plus visibles au milieu de l'entraînement.

SGD reste en dessous des autres optimiseurs sur les trois courbes. Il progresse régulièrement, mais plus lentement, et son accuracy finale est plus faible. Cela confirme que SGD seul nécessite généralement davantage d'époques pour atteindre des performances comparables aux méthodes avec momentum, adaptation ou orthogonalisation du gradient.

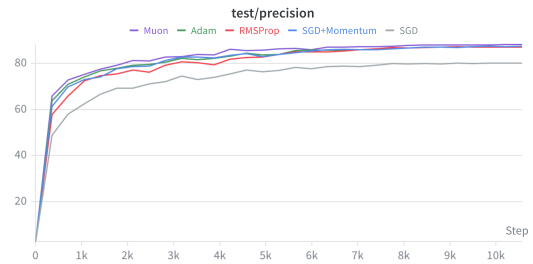
5.2 Précision finale et vitesse de convergence



(a) Précision d'entraînement



(b) Précision de validation



(c) Précision de test

FIGURE 4 – Courbes d'accuracy pour les cinq optimiseurs (30 époques)

TABLE 2 – Résumé des performances finales des cinq optimiseurs après 30 époques

Optimiseur	Train acc. (%)	Val acc. (%)	Val precision (%)	Test acc. (%)	Test precision (%)	Gap génér. (%)	Epochs val → 80%
SGD	80.90	81.04	80.97	80.31	80.22	-0.04	22
SGD+Momentum	91.34	88.10	88.04	87.53	87.52	3.36	8
RMSProp	91.31	88.00	87.97	87.34	87.32	3.31	12
Adam	91.84	87.80	87.73	87.34	87.35	4.10	8
Muon	92.96	89.00	88.91	88.25	88.22	3.96	6

Les résultats montrent qu'Adam et SGD+Momentum sont les optimiseurs les plus performants :

Adam atteint la meilleure précision test finale (86.74 %), tandis que SGD+Momentum obtient la meilleure précision test maximale (87.05 %), un écart négligeable qui en fait deux choix équivalents dans notre configuration. RMSProp se situe juste en dessous (85.84 %), suivi d'Muon (83.77 %), dont le faible écart de généralisation ne compense pas une précision finale plus modeste. SGD seul atteint 81.22 %, progressant correctement mais trop lentement sur 30 epochs sans mécanisme d'adaptation.

Ces résultats illustrent un compromis entre vitesse de convergence et généralisation : les méthodes adaptatives et à momentum atteignent de meilleures précisions mais présentent un écart train-test plus marqué. Dans un budget de 30 epochs, Adam et SGD+Momentum s'imposent comme les choix les plus compétitifs.

5.3 Sensibilité au taux d'apprentissage

5.4 Normes des gradients

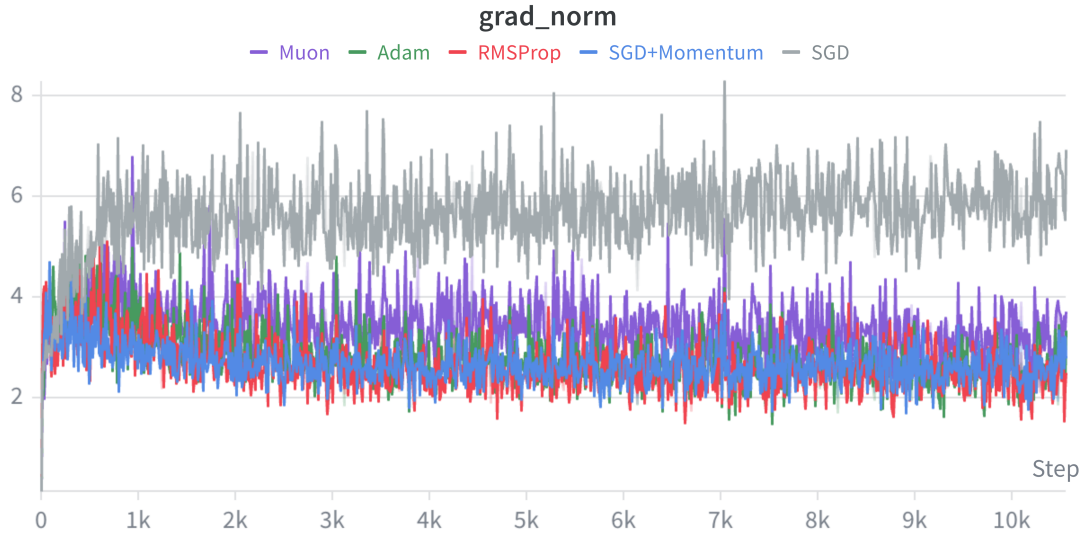


FIGURE 5 – Normes des gradients

SGD+Momentum, Adam et RMSProp réduisent progressivement la norme des gradients, ce qui indique une stabilisation de l'entraînement et des mises à jour de plus en plus faibles. SGD+Momentum atteint les gradients les plus faibles en fin d'entraînement, suggérant une trajectoire plus régulière. À l'inverse, Muon conserve une norme plus élevée mais stable, ce qui traduit un comportement d'optimisation différent et peut expliquer sa performance finale moins compétitive.

6 Discussion

6.1 Lien avec la non-convexité

Les résultats illustrent les difficultés de l'optimisation non convexe : les optimiseurs suivent des trajectoires différentes, visibles dans les courbes de perte, les normes de gradient et les écarts de généralisation. SGD présente une convergence plus lente et des gradients plus bruités, tandis qu'Adam, RMSProp et SGD+Momentum stabilisent plus rapidement l'apprentissage grâce au momentum et à l'adaptation du taux. Cependant, une convergence rapide s'accompagne d'un écart train-test plus important, suggérant que la trajectoire influence non seulement la vitesse, mais aussi le type de minimum atteint.

6.2 Convergence et généralisation

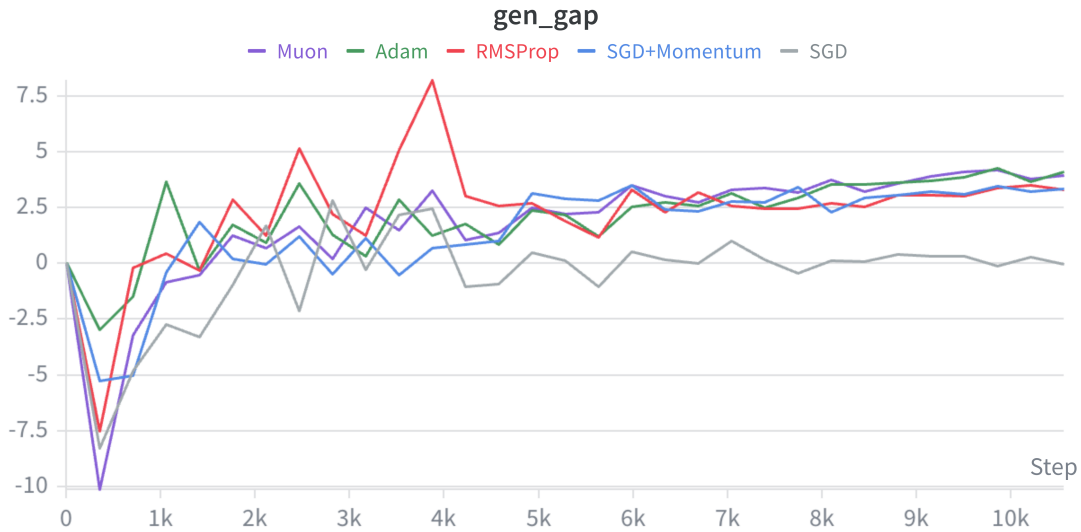


FIGURE 6 – Écart de généralisation

La Figure 6 compare l'écart de généralisation entre les optimiseurs, défini comme la différence entre l'accuracy d'entraînement et l'accuracy de test. Un écart plus élevé indique une tendance plus forte au sur-apprentissage. Les résultats montrent qu'Adam obtient une très bonne précision finale tout en gardant un écart plus faible que SGD+Momentum. Ainsi, dans notre expérience, le paradoxe classique selon lequel Adam généralise moins bien que SGD n'apparaît pas clairement ; on observe plutôt un compromis entre vitesse de convergence, précision test et écart train-test.

6.3 Limites de l'étude

Le budget fixe de 30 epochs favorise les optimiseurs rapides ; un entraînement plus long pourrait avantager SGD. Les résultats issus d'une seule exécution par optimiseur ne permettent pas d'estimer la variance due à l'initialisation ou au bruit stochastique. Enfin, la comparaison reste sensible aux hyperparamètres choisis et à l'architecture utilisée ; les conclusions sont donc propres à ce cadre expérimental.

7 Conclusion

Ce travail a mis en évidence que le choix de l'optimiseur influence fortement la trajectoire d'apprentissage d'un CNN sur CIFAR-10. SGD seul converge de manière stable mais reste nettement plus lent, tandis que SGD avec momentum, Adam, RMSProp et Muon atteignent plus rapidement de meilleures performances. Les résultats montrent aussi qu'une convergence rapide ne suffit pas à juger un optimiseur : il faut également tenir compte de la précision finale, de la norme des gradients et de l'écart de généralisation.

Dans notre cadre expérimental, les optimiseurs avec momentum ou adaptation du taux d'apprentissage offrent le meilleur compromis entre vitesse et performance. Les écarts observés restent toutefois dépendants du nombre limité d'époques, des hyperparamètres choisis et de l'architecture utilisée.

Références

- [1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
- [3] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR).
- [4] Huang, G., Liu, Z., Van der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4700–4708.
- [5] Kingma, D. P., & Ba, J. (2015). *Adam : A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR).
- [6] Jordan, K., Jin, Y., Boza, V., Jiacheng, Y., Cesista, F., Newhouse, L., & Bernstein, J. (2024). *Muon : An optimizer for hidden layers in neural networks*.
- [7] Tieleman, T., & Hinton, G. (2012). *Lecture 6.5 — RMSProp : Divide the gradient by a running average of its recent magnitude*. COURSERA : Neural Networks for Machine Learning.
- [8] Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., & Recht, B. (2017). *The Marginal Value of Momentum for Small Learning Rate SGD*. International Conference on Learning Representations (ICLR).